ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ «ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
МОСКОВСКИЙ ИНСТИТУТ ЭЛЕКТРОНИКИ И МАТЕМАТИКИ им. А.Н. ТИХОНОВА

ОТЧЁТ

по лабораторной работе №1

по дисциплине «Параллельные вычисления»

«Освоение векторизации»

Вариант 2. Транспонирование плотных матриц больших размерностей

Выполнил:
студент группы СКБ221
Романенко Д. А.

Проверил:
Байдин Г. С.

Москва 2026

СОДЕРЖАНИЕ

1 Постановка задачи....3
2 Вычислительная система и компилятор....4
3 Исходная программа и оптимизация ключами компилятора....5
4 Полуавтоматическая векторизация....6
4.1 Директива omp simd и restrict....7
4.2 Блочный обход....7
4.3 Микроблок с полной развёрткой....7
4.4 Дополнение строк матрицы....8
4.5 Подбор размера блока и микроблока....9
5 Ручная векторизация....10
5.1 Ядро 2×2....10
5.2 Блочный обход....11
5.3 Ядро 4×4....11
5.4 Предвыборка данных....11
5.5 Ядро 8×8....12
5.6 Векторизация на SME....12
6 Сводные результаты и сравнение с теоретическими оценками....13
7 Фрагменты кода на ассемблере....16
Заключение....18
Приложение А. Листинг программы....20

1 Постановка задачи

Задача работы состоит в освоении средств векторизации программ на языке C и в сравнении времени работы нескольких вариантов одной и той же программы: исходной программы без оптимизации, программы, оптимизированной только ключами компилятора, программы, векторизованной полуавтоматически (директивами компилятора и небольшими правками кода), и программы, векторизованной вручную с помощью встроенных функций (intrinsics) векторного расширения процессора.

По варианту 2 вычисляется транспонированная матрица $B = A^T$, то есть B[j][i] = A[i][j] для всех i, j от 0 до N − 1. Матрица квадратная, плотная, элементы типа double (8 байт), хранение по строкам. Основной размер N = 8192, при нём каждая матрица занимает 512 МБ, что во много раз больше всех кэшей процессора. Дополнительно замеры проводились для N = 8000, чтобы отделить влияние того, что 8192 является степенью двойки.

Арифметических операций в задаче нет: программа только читает N^2 чисел и записывает N^2 чисел, поэтому её скорость определяется работой подсистемы памяти. Основная трудность в том, что одна из двух матриц всегда проходится по столбцам с шагом N·8 байт, и при наивном обходе почти каждое обращение к ней приходится на новую строку кэша. Векторизация здесь даёт выигрыш только вместе с изменением порядка обхода: нужно читать и писать память непрерывными кусками, а перестановку элементов делать в регистрах.

Время измерялось только для самого транспонирования, без выделения и заполнения памяти; для замера использовалась функция clock_gettime (часы CLOCK_MONOTONIC). Каждая конфигурация запускалась 3 раза по 7 повторов, в таблицах приведено наименьшее время. После замеров результат каждого варианта полностью сравнивался с исходной матрицей, все варианты дали верный результат. Скорость в ГБ/с считалась как $2 \cdot N^2 \cdot 8$ байт (чтение A и запись B), делённое на время.

2 Вычислительная система и компилятор

Все замеры выполнены на ноутбуке MacBook Pro с процессором Apple M4 Pro при работе от сети. Характеристики получены командами sysctl, sw_vers и clang --version и приведены в таблице 1.

Таблица 1 — Характеристики вычислительной системы

Параметр	Значение
Процессор	Apple M4 Pro, архитектура ARMv9 (AArch64)
Ядра	12: 8 производительных (P) и 4 энергоэффективных (E)
Кэш L1 данных	128 КБ на P-ядро, 64 КБ на E-ядро
Кэш L2	16 МБ на кластер P-ядер, 4 МБ на кластер E-ядер
Строка кэша	128 байт
Размер страницы памяти	16 КБ
Оперативная память	24 ГБ
Векторные расширения	NEON (Advanced SIMD, 128 бит); SME/SME2 (512 бит в режиме streaming SVE); SVE вне streaming-режима нет
Операционная система	macOS 27.0
Компилятор	Apple clang 17.0.0 (clang-1700.3.19.1)

Программа однопоточная. Планировщик macOS отдаёт такую нагрузку производительному ядру, поэтому для оценок используются параметры P-ядра. У M4 Pro нет расширений AVX, основное векторное расширение — NEON с регистрами по 128 бит, в которые помещается 2 числа double. Кроме него процессор поддерживает SME (Scalable Matrix Extension) — самое новое из доступных расширений. В режиме streaming SVE длина вектора равна 512 битам (8 чисел double), а матричный регистр ZA делится на тайлы 8×8 double. Ключ -mcpu=native в Apple clang 17 SME не включает, поэтому SME-вариант собирался с ключом -mcpu=apple-m4.

Исходные тексты, скрипт сборки и замеров run_all.sh и полный вывод замеров results.txt находятся в каталоге task1 репозитория. Полные листинги программы приведены в приложении А.

3 Исходная программа и оптимизация ключами компилятора

Исходная программа написана «в лоб» (листинг 1): внешний цикл идёт по строкам A, внутренний — по столбцам. Матрица A читается последовательно, а запись в B идёт по столбцу с шагом N·8 байт. Матрицы хранятся с ведущей размерностью ld: элемент (i, j) лежит по адресу i·ld + j. По умолчанию ld = N; параметр понадобится в разделе 4.4. Память выделяется функцией aligned_alloc с выравниванием на 128 байт — по размеру строки кэша M4.

Листинг 1 — Функция транспонирования в исходной программе

```
void transpose_naive(const double *a, double *b, size_t n, size_t ld)
{
    for (size_t i = 0; i < n; i++)
        for (size_t j = 0; j < n; j++)
            b[j * ld + i] = a[i * ld + j];
}
```

Исходная программа собиралась с разными уровнями оптимизации и ключами под конкретный процессор. Результаты приведены в таблице 2.

Таблица 2 — Время исходной программы при разных ключах компилятора

Ключи	N = 8192, с	Ускорение к -O0	N = 8000, с	Ускорение к -O0
-O0	0,2652	1,00	0,2152	1,00
-O1	0,2889	0,92	0,1232	1,75
-O2	0,2934	0,90	0,1215	1,77
-O3	0,2932	0,90	0,1212	1,78
-O3 -mcpu=native	0,2932	0,90	0,1209	1,78
-O3 -mcpu=apple-m4	0,2934	0,90	0,1211	1,78
-O3 -ffast-math -mcpu=native -funroll-loops	0,2937	0,90	0,1204	1,79

При N = 8000 переход от -O0 к -O1 ускоряет программу в 1,75 раза: убираются лишние загрузки и сохранения переменных цикла в стек. Дальнейшие уровни (-O2, -O3), ключи -mcpu=native и -mcpu=apple-m4, -ffast-

math и -funroll-loops время почти не меняют, разница между всеми оптимизированными вариантами не более 2 %. Ключ -ffast-math ни на что не влияет, так как вычислений с плавающей точкой в программе нет.

При N = 8192 картина другая. Все оптимизированные версии работают около 0,29 с, а версия -O0 даже немного быстрее — 0,2652 с. Время здесь почти полностью уходит на ожидание памяти: шаг записи в B равен 8192·8 = 64 КБ, такие адреса попадают в одни и те же наборы кэша и вытесняют друг друга (конфликтные промахи). Поэтому сокращение числа команд ничего не даёт, а время при N = 8192 в 2,4 раза больше, чем при N = 8000, хотя данных больше всего на 5 %. Почему -O0 оказался чуть быстрее, точно установить без профилировщика не удалось; вероятно, более редкая выдача записей немного снижает число конфликтов.

Из отчёта векторизатора clang (ключи -Rpass=loop-vectorize, -Rpass-analysis=loop-vectorize) видно, что внутренний цикл в общем случае не векторизуется: «the cost-model indicates that vectorization is not beneficial». Сообщение «vectorized loop (vectorization width: 2)» относится к копии цикла, которую компилятор создаёт для частного случая ld = 1, когда запись идёт подряд. При N > 1 эта ветка не выполняется, и работает скалярный цикл из пар команд ldr d0 / str d0 (раздел 7). Таким образом, одними ключами компилятора задачу ускорить нельзя, нужно менять порядок обхода памяти. Для всех следующих вариантов использовались ключи -O3 -mcpu=native (для SME — -O3 -mcpu=apple-m4).

4 Полуавтоматическая векторизация

Полуавтоматическая версия (файл transpose_simd.c) собиралась командой clang -O3 -mcpu=native -fopenmp-simd. Ключ -fopenmp-simd включает обработку директив #pragma omp simd без подключения библиотеки OpenMP. Указатели помечены restrict, чтобы компилятор знал, что A и B не пересекаются. Ниже описаны этапы в том порядке, в котором они делались, а времена сведены в таблицу 3.

4.1 Директива omp simd и restrict

Сначала циклы были переставлены так, чтобы внутренний цикл шёл по строке B, то есть запись стала непрерывной, а чтение A — с шагом. На внутренний цикл поставлена директива #pragma omp simd. Компилятор сообщает, что цикл векторизован с шириной 2, но в ассемблере видно, что векторная копия цикла (ldp q / stp q) снова построена только для частного случая ld = 1, а при настоящем шаге выполняется скалярный цикл ldr d0 / str d0. Команд сбора (gather) с произвольным шагом в NEON нет, и компилятор посчитал сборку вектора из отдельных загрузок невыгодной. Тем не менее время при N = 8192 уменьшилось с 0,2932 с до 0,1883 с — за счёт того, что непрерывная запись дешевле записи по столбцу. Чтение A по столбцу всё так же даёт промах кэша почти на каждое обращение.

4.2 Блочный обход

Чтобы строки кэша A и B использовались полностью, матрица разбивается на блоки BS×BS и транспонирование выполняется поблочно. Пока обрабатывается блок, нужные строки кэша обеих матриц остаются в кэше. Внутри блока остался цикл с omp simd, который, как и в 4.1, в общем случае выполняется скалярно. Время при N = 8192 уменьшилось до 0,0771 с, то есть ещё в 2,44 раза.

4.3 Микроблок с полной развёрткой

Внутри блока BS×BS выделяются микроблоки TS×TS с размером-константой. Оба цикла микроблока полностью развёрнуты директивой #pragma clang loop unroll(full), а выравнивание указателей сообщается компилятору через __builtin_assume_aligned. После развёртки SLP-векторизатор объединяет соседние обращения: из каждой строки A и каждой строки B за раз берётся по TS чисел. Лучший результат дал микроблок 8×8 (64 байта, половина строки кэша), время при N = 8192 — 0,0699 с.

По ассемблеру видно (раздел 7), что clang и здесь не строит транспонирование из векторных перестановок и не использует векторные регистры: блок 8×8 переносится парными скалярными командами ldp d / stp d, каждая из которых читает или пишет два соседних числа (16 байт). То есть полуавтоматическая векторизация в этой задаче свелась к объединению обращений к памяти, а настоящие векторные команды появились только в ручной версии. Для задачи, ограниченной памятью, и этого оказывается достаточно.

4.4 Дополнение строк матрицы

Даже с блоками время при N = 8192 оставалось в 3,3 раза больше, чем при N = 8000 (0,0699 с против 0,0212 с). Причина та же — конфликтные промахи: строки блока отстоят друг от друга на 64 КБ, и все они попадают в один набор кэша L1. Поэтому к каждой строке матрицы было добавлено 8 неиспользуемых элементов (ld = N + 8, то есть 64 байта). Шаг между строками перестаёт быть степенью двойки, и строки блока распределяются по разным наборам. Время микроблочной версии упало до 0,0230 с, то есть в 3,04 раза. Для сравнения: исходная программа с ld = N + 8 работает 0,1010 с вместо 0,2932 с. Дополнение требует лишь 0,10 % дополнительной памяти.

Таблица 3 — Время полуавтоматически векторизованной программы, с

Вариант	N = 8192	N = 8192, ld = N + 8	N = 8000
исходная, -O3 -mcpu=native	0,2932	0,1010	0,1209
omp simd по строкам B (4.1)	0,1883	0,1563	0,1566
блоки BS×BS + omp simd (4.2)	0,0771	0,0731	0,0650
блоки + микроблок 8×8 (4.3)	0,0699	0,0230	0,0212

Здесь и далее BS = 256, если не сказано иное. Для варианта omp simd по строкам с ld = N + 8 разброс между повторами был большим (среднее

время заметно выше минимального), этот вариант чувствителен к размещению страниц в памяти.

4.5 Подбор размера блока и микроблока

Размер блока BS подбирался при N = 8192, ld = N + 8 (таблица 4), размер микроблока TS — при BS = 256 (таблица 5). Для сравнения в таблицу 4 включены также ручные варианты из раздела 5.

Таблица 4 — Зависимость времени от размера блока BS (N = 8192, ld = N + 8), с

BS	Блоки + микроблок 8×8	NEON 8×8 + блоки	SME, тайл 8×8
16	0,0416	0,0450	0,0331
32	0,0392	0,0398	0,0338
64	0,0252	0,0260	0,0312
128	0,0247	0,0246	0,0302
256	0,0215	0,0272	0,0297

Таблица 5 — Зависимость времени от размера микроблока TS (N = 8192, ld = N + 8, BS = 256)

TS	Время, с
2	0,0266
4	0,0248
8	0,0229

Маленькие блоки (16 и 32) работают заметно хуже: блок 16×16 занимает в строке матрицы всего 128 байт, и на каждый блок приходится много переходов между далёкими участками памяти. С ростом блока время уменьшается; начиная с BS = 64 различия между размерами блока невелики и для NEON-версии сравнимы с разбросом между запусками. Для полуавтоматической версии лучший результат дал BS = 256, он и использован в основных замерах. Для SME размер блока влияет слабо. Из микроблоков

лучшим оказался 8×8: он берёт из строки кэша 64 байта за раз, а 2×2 и 4×4 — только 16 и 32 байта.

Таблицы 3 и 4 получены в разных сериях запусков, поэтому одна и та же конфигурация (микроблок 8×8, BS = 256, ld = N + 8) имеет в них немного разное время: 0,0230 и 0,0215 с. Это разброс между запусками программы, связанный, по-видимому, с тем, как ОС размещает страницы матриц в физической памяти. Внутри одной таблицы варианты сравнивались в одинаковых условиях.

5 Ручная векторизация

Ручная векторизация выполнена на встроенных функциях NEON из заголовка arm_neon.h (файл transpose_neon.c), а в разделе 5.6 — на функциях SME из arm_sme.h (файл transpose_sme.c). Каждый этап оформлен отдельной функцией, и все этапы замеряются в одном запуске программы. Результаты сведены в таблицу 6. Там же приведено время копирования того же объёма функцией memcpy: транспонирование не может работать быстрее копирования, поэтому memcpy служит практической оценкой предела.

5.1 Ядро 2×2

Вектор NEON вмещает два числа double, поэтому базовое ядро транспонирует блок 2×2. Две строки блока загружаются командами vld1q_f64, затем команды vtrn1q_f64 и vtrn2q_f64 составляют из них два столбца: из векторов (a00, a01) и (a10, a11) получаются (a00, a10) и (a01, a11). Эти векторы записываются в две строки B командами vst1q_f64 (листинг 2). Матрица обходится такими блоками подряд, без разбиения на большие блоки.

Листинг 2 — Ядро 2×2 на NEON

```
static inline void kernel2(const double *a, double *b, size_t ld)
{
    float64x2_t r0 = vld1q_f64(a);
    float64x2_t r1 = vld1q_f64(a + ld);
    vst1q_f64(b,      vtrn1q_f64(r0, r1));
    vst1q_f64(b + ld, vtrn2q_f64(r0, r1));
}
```

При ld = N + 8 время составило 0,0523 с против 0,1010 с у исходной программы с тем же ld, то есть ускорение 1,93 раза. Это близко к теоретическому значению 2 для векторов из двух элементов: число команд чтения и записи уменьшилось вдвое, и вдвое реже происходит переход на новую строку B.

5.2 Блочный обход

Ядро 2×2 было помещено в блочный обход BS×BS, как в разделе 4.2. При ld = N + 8 время уменьшилось до 0,0265 с. Без дополнения строк (ld = N) выигрыш почти пропадает (0,1481 → 0,1446 с): строки блока конфликтуют в кэше, и блочный обход не может удержать их там.

5.3 Ядро 4×4

Чтобы реже обращаться к памяти и больше данных держать в регистрах, ядро было расширено до 4×4. Читаются 4 строки по 4 числа (8 векторных регистров), перестановки выполняются в регистрах, а затем 4 строки B записываются по 32 байта. Из каждой строки кэша за один заход используется вдвое больше данных, чем в ядре 2×2. Время при ld = N + 8 — 0,0235 с.

5.4 Предвыборка данных

Ядро 4×4 было переписано на парные загрузку и запись (vld1q_f64_x2 и vst1q_f64_x2, по 32 байта), и в цикл добавлена программная предвыборка строк A на несколько микроблоков вперёд (__builtin_prefetch). Время не уменьшилось: 0,0245 с против 0,0235 с при ld = N + 8 и 0,0256 с против 0,0230 с при N = 8000. Аппаратный механизм предвыборки M4 сам распознаёт последовательное чтение строк, и дополнительные команды только занимают конвейер. Программная предвыборка в следующем этапе не использовалась.

5.5 Ядро 8×8

Ядро 8×8 составлено из четырёх ядер 4×4 с парными загрузками. Из каждой строки кэша A и B за один заход берётся 64 байта, как и в лучшем полуавтоматическом варианте. Время при ld = N + 8 составило 0,0247 с против 0,0235 с у ядра 4×4, при N = 8000 — 0,0232 с против 0,0230 с. Ядра 4×4 и 8×8 дают практически одинаковое время: разница между ними не превышает разброса между запусками (в предыдущих сериях замеров ядро 8×8 было немного быстрее ядра 4×4, в итоговой — наоборот). Видимо, уже ядро 4×4 загружает подсистему памяти до предела, который допускает такой порядок обхода. В ассемблере (раздел 7) видно, что ядро компилируется ровно в задуманные команды: ld1.2d по два регистра, zip1.2d/zip2.2d (так clang записывает trn1/trn2 для 64-битных элементов) и st1.2d.

5.6 Векторизация на SME

Расширение SME позволяет транспонировать блок вообще без перестановок. Матричный регистр ZA делится на 8 тайлов ZA0.D–ZA7.D по 8×8 чисел double. Строку тайла можно загрузить «горизонтально» командой svld1_hor_za64, а выгрузить «вертикально» командой svst1_ver_za64, то есть по столбцу. Загрузив 8 строк блока A и выгрузив 8 столбцов тайла в строки B, получаем транспонированный блок 8×8 (листинг 3). Функцию с этими командами нужно пометить двумя атрибутами: __arm_locally_streaming и __arm_new("za"). Тогда компилятор сам включает streaming-режим и регистр ZA в начале функции (smstart) и выключает в конце (smstop), поэтому переключение происходит один раз за всё транспонирование.

Листинг 3 — Транспонирование блока 8×8 через тайл ZA0

```
svbool_t pg = svptrue_b64();
for (uint32_t r = 0; r < 8; r++)
    svld1_hor_za64(0, r, pg, pa + r * ld);   /* строка A -> строка тайла */
for (uint32_t c = 0; c < 8; c++)
    svst1_ver_za64(0, c, pg, pb + c * ld);   /* столбец тайла -> строка B */
```

Вариант с одним тайлом и блочным обходом дал 0,0322 с при ld = N, 0,0298 с при ld = N + 8 и 0,0281 с при N = 8000. Вторым этапом был

опробован блок 16×16 из четырёх тайлов, чтобы в B писалось сразу по 128 байт (целая строка кэша). Он оказался не быстрее: 0,0365, 0,0311 и 0,0330 с, поэтому лучшим SME-вариантом остаётся тайл 8×8.

Интересно, что SME-вариант почти не чувствителен к дополнению строк: при ld = N он в 3,0 раза быстрее NEON-ядра 8×8. Обращения к памяти в streaming-режиме выполняет отдельный матричный блок процессора, общий для кластера ядер и подключённый к кэшу L2, а не к L1 ядра. По-видимому, поэтому конфликты в L1 на него и не влияют. Но при ld = N + 8 NEON быстрее (0,0247 с против 0,0298 с): по-видимому, у матричного блока больше задержка доступа к памяти, а теоретическое преимущество в ширине вектора (8 чисел против 2) в задаче без вычислений не используется.

Таблица 6 — Время программы с ручной векторизацией, с

Вариант	N = 8192	N = 8192, ld = N + 8	N = 8000
memcpy того же объёма (предел)	0,0082	0,0082	0,0078
NEON 2×2 (5.1)	0,1481	0,0523	0,0517
NEON 2×2 + блоки (5.2)	0,1446	0,0265	0,0259
NEON 4×4 + блоки (5.3)	0,1005	0,0235	0,0230
NEON 4×4 + предвыборка (5.4)	0,1008	0,0245	0,0256
NEON 8×8 + блоки (5.5)	0,0963	0,0247	0,0232
SME, тайл 8×8 + блоки (5.6)	0,0322	0,0298	0,0281
SME, 4 тайла 16×16 + блоки (5.6)	0,0365	0,0311	0,0330

Этапы 5.1–5.3 последовательно уменьшали время, переход к ядру 8×8 заметного выигрыша уже не дал. Этап с предвыборкой и вариант SME с четырьмя тайлами времени не уменьшили и в дальнейшем не использовались.

6 Сводные результаты и сравнение с теоретическими оценками

В таблице 7 приведены результаты последовательных оптимизаций при N = 8192. Ускорение считается относительно исходной программы,

собранной с -O0. Скорость — объём прочитанных и записанных данных, делённый на время.

Таблица 7 — Результаты последовательных оптимизаций (N = 8192)

Вариант	Время, с	Ускорение	ГБ/с
Исходная программа, -O0	0,2652	1,0	4,0
Исходная программа, -O3 -mcpu=native	0,2932	0,9	3,7
То же, ld = N + 8	0,1010	2,6	10,6
omp simd по строкам B, ld = N + 8	0,1563	1,7	6,9
Блоки + omp simd, ld = N + 8	0,0731	3,6	14,7
Блоки + микроблок 8×8, ld = N + 8	0,0230	11,5	46,7
NEON 2×2, ld = N + 8	0,0523	5,1	20,5
NEON 2×2 + блоки, ld = N + 8	0,0265	10,0	40,5
NEON 4×4 + блоки, ld = N + 8	0,0235	11,3	45,7
NEON 8×8 + блоки, ld = N + 8	0,0247	10,7	43,5
SME, тайл 8×8 + блоки, ld = N + 8	0,0298	8,9	36,0
memcpy того же объёма	0,0082	32,3	130,9

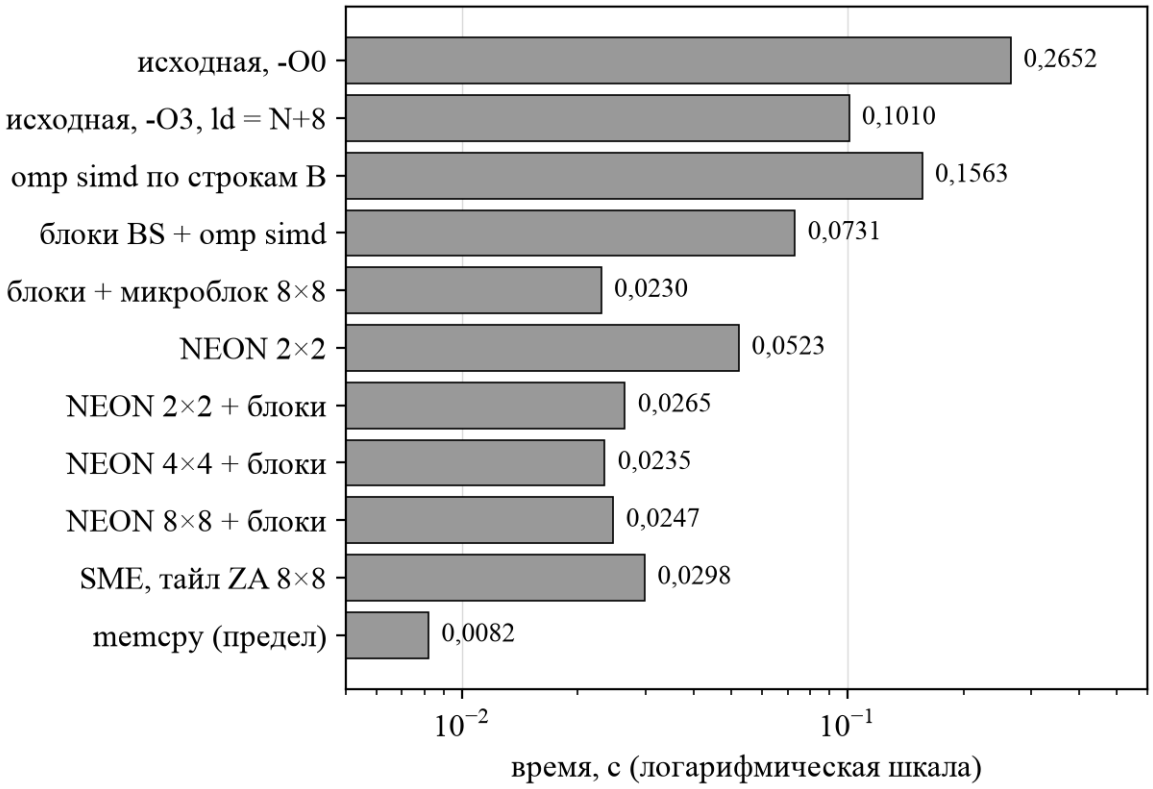


Рисунок 1 — Время работы вариантов программы при N = 8192

Теоретическое ускорение от векторизации на NEON равно 2 (128 бит / 64 бита), на SME — 8 (512 бит / 64 бита). Эта оценка относится к числу команд, а не ко времени. Для NEON она подтвердилась при сравнении скалярного и векторного вариантов с одинаковым обходом: 0,1010 с и 0,0523 с, ускорение 1,93. Для SME восьмикратного выигрыша нет, потому что в задаче нет вычислений и время определяется памятью, а не числом команд.

Практическую верхнюю границу задаёт копирование того же объёма: memcpy выполняется за 0,0082 с (130,9 ГБ/с). Лучшие варианты транспонирования работают 0,0230 с, то есть в 2,8 раза медленнее копирования. Разница объясняется тем, что копирование идёт строго последовательно и хорошо предсказывается аппаратной предвыборкой, а транспонирование одновременно работает с 8 строками A и 8 строками B, находящимися далеко друг от друга, и использует каждую строку кэша B за несколько заходов.

Итоговое ускорение относительно исходной программы составило 11,5 раза. Основной вклад дали изменение порядка обхода (блоки) и устранение

конфликтных промахов (дополнение строк); сама векторизация дала около двух раз, как и ожидалось теоретически. Полуавтоматическая версия с микроблоком 8×8 и лучшая ручная версия на NEON (ядро 4×4) показали близкое время (0,0230 и 0,0235 с).

7 Фрагменты кода на ассемблере

Ассемблерный код получен командой clang -O3 -mcpu=native -S (для SME — -mcpu=apple-m4), полные файлы лежат в каталоге task1/asm. В листинге 4 приведён внутренний цикл исходной программы. Каждый элемент переносится отдельной парой скалярных команд ldr d0 / str d0, адрес записи увеличивается на ld·8 байт.

Листинг 4 — Внутренний цикл исходной программы (-O3)

```
LBB0_38:
    ldr    d0, [x0, x10, lsl #3]
    str    d0, [x11]
    add    x10, x10, #1
    add    x11, x11, x9
    cmp    x2, x10
    b.ne    LBB0_38
```

В листинге 5 показан фрагмент развёрнутого микроблока 8×8 полуавтоматической версии. Компилятор объединил соседние обращения в парные скалярные команды ldp d / stp d, но векторных перестановок не построил.

Листинг 5 — Фрагмент микроблока 8×8 полуавтоматической версии

```
LBB2_8:
    mov    x15, x22
    ldr    d0, [x15, #-32]!
    add    x26, x15, x12
    add    x25, x26, x12
    ldp    d1, d2, [x26]
    stp    d0, d1, [x23]
    add    x30, x25, x12
    add    x27, x30, x12
    ldp    d0, d1, [x25]
    ldp    d3, d4, [x30]
    stp    d0, d3, [x23, #16]
    add    x15, x27, x12
    add    x28, x15, x12
    ldp    d0, d3, [x27]
    ldp    d5, d6, [x15]
    stp    d0, d5, [x23, #32]
    add    x3, x28, x12
    ldp    d0, d5, [x28]
    ldp    d7, d16, [x3]
    stp    d0, d7, [x23, #48]
    ldp    d0, d7, [x22, #-24]
    add    x2, x23, x12
    add    x11, x2, x12
```

В листинге 6 приведена первая четверть тела внутреннего цикла NEON-версии с ядром 8×8 (одно ядро 4×4). Видны загрузки ld1.2d по два регистра (32 байта), перестановки zip1.2d/zip2.2d и записи st1.2d с шагом ld·8 байт.

Листинг 6 — Ядро 4×4 в составе ядра 8×8 (NEON)

```
LBB4_8:
    add    x16, x22, x1
    ld1.2d    { v0, v1 }, [x16], #32
    add    x19, x23, x1
    ld1.2d    { v2, v3 }, [x19], #32
    add    x13, x24, x1
    ld1.2d    { v4, v5 }, [x13], #32
    add    x7, x25, x1
    ld1.2d    { v6, v7 }, [x7], #32
    zip1.2d    v16, v0, v2
    zip1.2d    v17, v4, v6
    st1.2d    { v16, v17 }, [x5], x10
    zip2.2d    v16, v0, v2
    zip2.2d    v17, v4, v6
    add    x9, x8, x15
    st1.2d    { v16, v17 }, [x9], x15
    zip1.2d    v16, v1, v3
    zip1.2d    v17, v5, v7
    zip2.2d    v0, v1, v3
    st1.2d    { v16, v17 }, [x9], x15
    zip2.2d    v1, v5, v7
    st1.2d    { v0, v1 }, [x9], x15
    ...
```

В листинге 7 приведён внутренний цикл SME-версии: восемь горизонтальных загрузок строк в тайл za0h и восемь вертикальных выгрузок столбцов za0v.

Листинг 7 — Внутренний цикл SME-версии (тайл ZA0, 8×8)

```
LBB1_10:
    mov    w15, #0
    ld1d    {za0h.d[w15, 0]}, p0/z, [x22]
    add    x25, x22, x10
    add    x26, x25, x10
    ld1d    {za0h.d[w13, 0]}, p0/z, [x25]
    ld1d    {za0h.d[w14, 0]}, p0/z, [x26]
    add    x25, x26, x10
    add    x26, x25, x10
    mov    w12, #3
    ld1d    {za0h.d[w12, 0]}, p0/z, [x25]
    mov    w12, #4
    ld1d    {za0h.d[w12, 0]}, p0/z, [x26]
    add    x25, x26, x10
    add    x26, x25, x10
    mov    w12, #5
    ld1d    {za0h.d[w12, 0]}, p0/z, [x25]
    mov    w12, #6
    ld1d    {za0h.d[w12, 0]}, p0/z, [x26]
    add    x25, x26, x10
    mov    w12, #7
    ld1d    {za0h.d[w12, 0]}, p0/z, [x25]
    st1d    {za0v.d[w15, 0]}, p0, [x23]
    add    x15, x23, x10
    add    x25, x15, x10
    st1d    {za0v.d[w13, 0]}, p0, [x15]
    st1d    {za0v.d[w14, 0]}, p0, [x25]
    add    x15, x25, x10
    add    x25, x15, x10
    mov    w12, #3
    st1d    {za0v.d[w12, 0]}, p0, [x15]
    mov    w12, #4
    st1d    {za0v.d[w12, 0]}, p0, [x25]
    add    x15, x25, x10
    add    x25, x15, x10
    mov    w12, #5
    st1d    {za0v.d[w12, 0]}, p0, [x15]
    mov    w12, #6
    st1d    {za0v.d[w12, 0]}, p0, [x25]
    add    x15, x25, x10
    mov    w12, #7
    st1d    {za0v.d[w12, 0]}, p0, [x15]
    add    x24, x24, #8
    add    x23, x23, x9
    add    x22, x22, #64
    cmp    x24, x7
    b.lo    LBB1_10
```

Заключение

В работе исследована векторизация транспонирования плотной матрицы double размером 8192×8192 на процессоре Apple M4 Pro. Исходная

программа работает 0,2652 с, и ключи компилятора её не ускоряют: внутренний цикл с записью по столбцу компилятор не векторизует, а время определяется промахами кэша.

Полуавтоматическая векторизация (omp simd, restrict, блочный обход, микроблок 8×8 с полной развёрткой) вместе с дополнением строк до ld = N + 8 уменьшила время до 0,0230 с. При этом настоящих векторных команд clang так и не построил: циклы с omp simd в общем случае остались скалярными, а микроблок переносится парными скалярными командами ldp d / stp d. Ручная векторизация на NEON (лучшее ядро — 4×4) дала 0,0235 с, то есть практически столько же. Общее ускорение относительно исходной программы — 11,5 раза, а до предела, заданного копированием памяти, остаётся примерно 2,8 раза.

Векторизация сама по себе дала ускорение 1,93, что соответствует теоретическому значению 2 для NEON. Наибольший же выигрыш дали изменения работы с памятью: блочный обход и устранение конфликтных промахов при размере матрицы, равном степени двойки (ускорение в 3,0 раза для одной и той же программы). Программная предвыборка не помогла. Самое новое расширение SME позволяет транспонировать блок без перестановок и не страдает от конфликтов в L1, но в этой задаче оно уступило NEON из-за большей задержки доступа к памяти.

Получившаяся блочная структура программы подходит для следующих заданий: блоки BS×BS независимы и могут обрабатываться разными потоками OpenMP, а при распределении по процессам MPI удобно делить матрицу на полосы строк.

Приложение А. Листинг программы

Ниже приведены полные тексты файлов программы.

Листинг А.1 — common.h. Общая часть: память, проверка, замер времени

```
/*
 * common.h -- общая часть для всех вариантов программы транспонирования:
 * выделение памяти, заполнение, проверка результата, замер времени.
 */
#ifndef COMMON_H
#define COMMON_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define ALIGN 128           /* выравнивание под строку кэша Apple M4 (128 байт) */

static double wtime(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec * 1e-9;
}

/*
 * Матрица n x n хранится по строкам с ведущей размерностью ld >= n:
 * элемент (i, j) лежит в p[i * ld + j]. При ld = n это обычное плотное
 * хранение; ld = n + pad добавляет в конец каждой строки pad пустых
 * элементов, чтобы шаг между строками не был степенью двойки.
 */
static double *alloc_matrix(size_t n, size_t ld)
{
    size_t bytes = n * ld * sizeof(double);
    bytes = (bytes + ALIGN - 1) / ALIGN * ALIGN;
    double *p = aligned_alloc(ALIGN, bytes);
    if (!p) {
        fprintf(stderr, "не удалось выделить %zu байт\n", bytes);
        exit(1);
    }
    return p;
}

static void fill_matrix(double *a, size_t n, size_t ld)
{
    for (size_t i = 0; i < n; i++)
        for (size_t j = 0; j < n; j++)
            a[i * ld + j] = (double)(i * n + j);
}

/* возвращает число неверных элементов */
static size_t check_transpose(const double *a, const double *b,
                              size_t n, size_t ld)
{
    size_t bad = 0;
    for (size_t i = 0; i < n; i++)
        for (size_t j = 0; j < n; j++)
            if (b[j * ld + i] != a[i * ld + j])
                bad++;
    return bad;
}

typedef void (*transpose_fn)(const double *a, double *b, size_t n, size_t ld);

/*
 * Запускает fn reps раз и печатает минимальное и среднее время.
 * Перед замером матрица b заполняется нулями (чтобы страницы были
```

```
 * уже выделены ОС и не влияли на время).
 */
static double bench(const char *name, transpose_fn fn,
                    const double *a, double *b, size_t n, size_t ld, int reps)
{
    double tmin = 1e30, tsum = 0;
    memset(b, 0, n * ld * sizeof(double));
    for (int r = 0; r < reps; r++) {
        double t0 = wtime();
        fn(a, b, n, ld);
        double t = wtime() - t0;
        if (t < tmin) tmin = t;
        tsum += t;
    }
    size_t bad = check_transpose(a, b, n, ld);
    /* через память проходит 2 * n^2 * 8 байт (чтение A + запись B) */
    double gbs = 2.0 * n * n * sizeof(double) / tmin / 1e9;
    printf("%-28s  min %8.4f s  avg %8.4f s  %6.2f GB/s  %s\n",
           name, tmin, tsum / reps, gbs, bad ? "ОШИБКА" : "ok");
    fflush(stdout);
    return tmin;
}

static size_t arg_n(int argc, char **argv, size_t def)
{
    return argc > 1 ? (size_t)strtoull(argv[1], NULL, 10) : def;
}

static int arg_reps(int argc, char **argv, int def)
{
    return argc > 2 ? atoi(argv[2]) : def;
}

/* третий аргумент -- дополнение строки pad (ld = n + pad), по умолчанию 0 */
static size_t arg_ld(int argc, char **argv, size_t n)
{
    return n + (argc > 3 ? (size_t)strtoull(argv[3], NULL, 10) : 0);
}

#endif
```

Листинг А.2 — transpose_naive.c. Исходная программа

```
/*
 * transpose_naive.c -- исходная программа: транспонирование плотной
 * квадратной матрицы B = A^T "в лоб". Никаких правок под векторизацию.
 *
 * Запуск: ./transpose_naive [N] [повторы] [pad]
 */
#include "common.h"

void transpose_naive(const double *a, double *b, size_t n, size_t ld)
{
    for (size_t i = 0; i < n; i++)
        for (size_t j = 0; j < n; j++)
            b[j * ld + i] = a[i * ld + j];
}

int main(int argc, char **argv)
{
    size_t n = arg_n(argc, argv, 8192);
    int reps = arg_reps(argc, argv, 5);
    size_t ld = arg_ld(argc, argv, n);
    double *a = alloc_matrix(n, ld), *b = alloc_matrix(n, ld);
    fill_matrix(a, n, ld);
    printf("N = %zu, матрица %.0f МБ\n", n, n * n * 8.0 / (1 << 20));
    bench("naive", transpose_naive, a, b, n, ld, reps);
    free(a);
    free(b);
    return 0;
}
```

Листинг А.3 — transpose_simd.c. Полуавтоматическая векторизация

```
/*
 * transpose_simd.c -- полуавтоматическая векторизация транспонирования:
 * директивы OpenMP SIMD и clang loop, restrict, подсказки о выравнивании,
 * блочный (тайловый) обход и небольшие правки кода.
 *
 * Сборка: clang -O3 -mcpu=native -fopenmp-simd -DBS=256 -DTS=8 transpose_simd.c
 * Запуск: ./transpose_simd [N] [повторы] [pad]
 */
#include "common.h"

#ifndef BS
#define BS 256              /* размер внешнего блока (тайла) */
#endif
#ifndef TS
#define TS 8                /* размер микроблока: 8 double = 64 байта */
#endif

/* Этап 1: restrict + omp simd, внутренний цикл пишет B подряд */
void transpose_simd_rows(const double *restrict a, double *restrict b, size_t n,
size_t ld)
{
    for (size_t j = 0; j < n; j++) {
        #pragma omp simd
        for (size_t i = 0; i < n; i++)
            b[j * ld + i] = a[i * ld + j];
    }
}

/* Этап 2: блочный обход BS x BS, внутри блока omp simd */
void transpose_blocked(const double *restrict a, double *restrict b, size_t n, size_t
ld)
{
    for (size_t ii = 0; ii < n; ii += BS)
        for (size_t jj = 0; jj < n; jj += BS) {
            size_t ie = ii + BS < n ? ii + BS : n;
            size_t je = jj + BS < n ? jj + BS : n;
            for (size_t j = jj; j < je; j++) {
```

```
                #pragma omp simd
                for (size_t i = ii; i < ie; i++)
                    b[j * ld + i] = a[i * ld + j];
            }
        }
}

/*
 * Микроблок TS x TS с размерами-константами: оба цикла полностью
 * разворачиваются, после чего компилятор объединяет соседние обращения
 * к строкам A и B (clang делает из них парные команды ldp/stp).
 */
static inline void tile(const double *restrict a, double *restrict b, size_t ld)
{
    #pragma clang loop unroll(full)
    for (int j = 0; j < TS; j++) {
        #pragma clang loop unroll(full)
        for (int i = 0; i < TS; i++)
            b[j * ld + i] = a[i * ld + j];
    }
}

/* Этап 3: двухуровневые блоки (BS x BS из микроблоков TS x TS) */
void transpose_tiled(const double *restrict a, double *restrict b, size_t n, size_t
ld)
{
    a = __builtin_assume_aligned(a, ALIGN);
    b = __builtin_assume_aligned(b, ALIGN);
    size_t nt = n / TS * TS;
    for (size_t ii = 0; ii < nt; ii += BS)
        for (size_t jj = 0; jj < nt; jj += BS) {
            size_t ie = ii + BS < nt ? ii + BS : nt;
            size_t je = jj + BS < nt ? jj + BS : nt;
            for (size_t i = ii; i < ie; i += TS)
                for (size_t j = jj; j < je; j += TS)
                    tile(a + i * ld + j, b + j * ld + i, ld);
        }
    /* хвосты, если n не кратно TS */
    for (size_t i = 0; i < n; i++)
        for (size_t j = (i < nt ? nt : 0); j < n; j++)
            b[j * ld + i] = a[i * ld + j];
}

int main(int argc, char **argv)
{
    size_t n = arg_n(argc, argv, 8192);
    int reps = arg_reps(argc, argv, 5);
    size_t ld = arg_ld(argc, argv, n);
    double *a = alloc_matrix(n, ld), *b = alloc_matrix(n, ld);
    fill_matrix(a, n, ld);
    printf("N = %zu, BS = %d, TS = %d\n", n, BS, TS);
    bench("simd_rows (restrict+simd)", transpose_simd_rows, a, b, n, ld, reps);
    bench("blocked (BSxBS + simd)", transpose_blocked, a, b, n, ld, reps);
    bench("tiled (BSxBS, TSxTS SLP)", transpose_tiled, a, b, n, ld, reps);
    free(a);
    free(b);
    return 0;
}
```

Листинг A.4 — transpose_neon.c. Ручная векторизация на NEON

```
/*
 * transpose_neon.c -- ручная векторизация транспонирования на intrinsics
 * NEON (ARMv8, 128 бит = 2 double). Каждый этап оптимизации оформлен
 * отдельной функцией, все они замеряются в одном запуске вместе с
 * эталонным memcpy того же объёма.
 *
 * Сборка: clang -O3 -mcpu=native -DBS=256 transpose_neon.c
 * Запуск: ./transpose_neon [N] [повторы] [pad]
 */
```

```
#include "common.h"
#include <arm_neon.h>

#ifndef BS
#define BS 256              /* размер внешнего блока (тайла) */
#endif
#ifndef PF
#define PF 8                /* на сколько микроблоков вперёд делать prefetch */
#endif

/* хвосты: элементы, не попавшие в блоки по k (n не кратно k) */
static void tail(const double *a, double *b, size_t n, size_t ld, size_t k)
{
    size_t nk = n / k * k;
    for (size_t i = 0; i < n; i++)
        for (size_t j = (i < nk ? nk : 0); j < n; j++)
            b[j * ld + i] = a[i * ld + j];
}

/*
 * Ядро 2x2: две строки A читаются двумя векторами, перестановками
 * trn1/trn2 получаются два столбца, которые записываются строками B.
 *   r0 = (a00 a01), r1 = (a10 a11)
 *   trn1 -> (a00 a10), trn2 -> (a01 a11)
 */
static inline void kernel2(const double *a, double *b, size_t ld)
{
    float64x2_t r0 = vld1q_f64(a);
    float64x2_t r1 = vld1q_f64(a + ld);
    vst1q_f64(b,     vtrn1q_f64(r0, r1));
    vst1q_f64(b + ld, vtrn2q_f64(r0, r1));
}

/* Этап 1: ядро 2x2, простой обход всей матрицы */
void transpose_neon2(const double *restrict a, double *restrict b, size_t n, size_t
ld)
{
    size_t n2 = n / 2 * 2;
    for (size_t i = 0; i < n2; i += 2)
        for (size_t j = 0; j < n2; j += 2)
            kernel2(a + i * ld + j, b + j * ld + i, ld);
    tail(a, b, n, ld, 2);
}

/* Этап 2: ядро 2x2 + блочный обход BS x BS */
void transpose_neon2_blocked(const double *restrict a, double *restrict b, size_t n,
size_t ld)
{
    size_t n2 = n / 2 * 2;
    for (size_t ii = 0; ii < n2; ii += BS)
        for (size_t jj = 0; jj < n2; jj += BS) {
            size_t ie = ii + BS < n2 ? ii + BS : n2;
            size_t je = jj + BS < n2 ? jj + BS : n2;
            for (size_t i = ii; i < ie; i += 2)
                for (size_t j = jj; j < je; j += 2)
                    kernel2(a + i * ld + j, b + j * ld + i, ld);
        }
    tail(a, b, n, ld, 2);
}

/*
 * Ядро 4x4 из четырёх 2x2: читается 4 строки по 4 double (8 векторов),
 * все перестановки делаются в регистрах, записывается 4 строки B.
 * Каждая строка кэша A и B за одно обращение используется на 32 байта
 * вместо 16, обращений к памяти на элемент вдвое меньше, чем при
 * последовательном вызове 2x2 по одной паре строк.
 */
static inline void kernel4(const double *a, double *b, size_t ld)
{
    float64x2_t a0l = vld1q_f64(a),          a0h = vld1q_f64(a + 2);
    float64x2_t a1l = vld1q_f64(a + ld),     a1h = vld1q_f64(a + ld + 2);
    float64x2_t a2l = vld1q_f64(a + 2 * ld), a2h = vld1q_f64(a + 2 * ld + 2);
```

```
    float64x2_t a3l = vld1q_f64(a + 3 * ld), a3h = vld1q_f64(a + 3 * ld + 2);

    vst1q_f64(b,             vtrn1q_f64(a0l, a1l));
    vst1q_f64(b + 2,         vtrn1q_f64(a2l, a3l));
    vst1q_f64(b + ld,         vtrn2q_f64(a0l, a1l));
    vst1q_f64(b + ld + 2,     vtrn2q_f64(a2l, a3l));
    vst1q_f64(b + 2 * ld,     vtrn1q_f64(a0h, a1h));
    vst1q_f64(b + 2 * ld + 2, vtrn1q_f64(a2h, a3h));
    vst1q_f64(b + 3 * ld,     vtrn2q_f64(a0h, a1h));
    vst1q_f64(b + 3 * ld + 2, vtrn2q_f64(a2h, a3h));
}

/* Этап 3: ядро 4x4 + блочный обход BS x BS */
void transpose_neon4_blocked(const double *restrict a, double *restrict b, size_t n,
size_t ld)
{
    size_t n4 = n / 4 * 4;
    for (size_t ii = 0; ii < n4; ii += BS)
        for (size_t jj = 0; jj < n4; jj += BS) {
            size_t ie = ii + BS < n4 ? ii + BS : n4;
            size_t je = jj + BS < n4 ? jj + BS : n4;
            for (size_t i = ii; i < ie; i += 4)
                for (size_t j = jj; j < je; j += 4)
                    kernel4(a + i * ld + j, b + j * ld + i, ld);
        }
    tail(a, b, n, ld, 4);
}

/*
 * Ядро 4x4 с загрузкой/записью парами векторов (vld1q_f64_x2 /
 * vst1q_f64_x2 -> ldp q / stp q): 4 загрузки и 4 записи по 32 байта.
 */
static inline void kernel4x2(const double *a, double *b, size_t ld)
{
    float64x2x2_t r0 = vld1q_f64_x2(a);
    float64x2x2_t r1 = vld1q_f64_x2(a + ld);
    float64x2x2_t r2 = vld1q_f64_x2(a + 2 * ld);
    float64x2x2_t r3 = vld1q_f64_x2(a + 3 * ld);
    float64x2x2_t c;

    c.val[0] = vtrn1q_f64(r0.val[0], r1.val[0]);
    c.val[1] = vtrn1q_f64(r2.val[0], r3.val[0]);
    vst1q_f64_x2(b, c);
    c.val[0] = vtrn2q_f64(r0.val[0], r1.val[0]);
    c.val[1] = vtrn2q_f64(r2.val[0], r3.val[0]);
    vst1q_f64_x2(b + ld, c);
    c.val[0] = vtrn1q_f64(r0.val[1], r1.val[1]);
    c.val[1] = vtrn1q_f64(r2.val[1], r3.val[1]);
    vst1q_f64_x2(b + 2 * ld, c);
    c.val[0] = vtrn2q_f64(r0.val[1], r1.val[1]);
    c.val[1] = vtrn2q_f64(r2.val[1], r3.val[1]);
    vst1q_f64_x2(b + 3 * ld, c);
}

/* Этап 4: ядро 4x4 с парными загрузками + блочный обход + prefetch */
void transpose_neon4_pf(const double *restrict a, double *restrict b, size_t n, size_t
ld)
{
    size_t n4 = n / 4 * 4;
    for (size_t ii = 0; ii < n4; ii += BS)
        for (size_t jj = 0; jj < n4; jj += BS) {
            size_t ie = ii + BS < n4 ? ii + BS : n4;
            size_t je = jj + BS < n4 ? jj + BS : n4;
            for (size_t i = ii; i < ie; i += 4) {
                const double *ar = a + i * ld;
                for (size_t j = jj; j < je; j += 4) {
                    /* подкачиваем строки A на PF микроблоков вперёд */
                    __builtin_prefetch(ar + j + 4 * PF, 0, 0);
                    __builtin_prefetch(ar + ld + j + 4 * PF, 0, 0);
                    __builtin_prefetch(ar + 2 * ld + j + 4 * PF, 0, 0);
                    __builtin_prefetch(ar + 3 * ld + j + 4 * PF, 0, 0);
                    kernel4x2(ar + j, b + j * ld + i, ld);
```

```
                }
            }
        }
    tail(a, b, n, ld, 4);
}

/*
 * Ядро 8x8 из четырёх ядер 4x4: из каждой строки кэша A и B за один
 * заход используется 64 байта (половина 128-байтной строки M4).
 */
static inline void kernel8(const double *a, double *b, size_t ld)
{
    kernel4x2(a,             b,             ld);
    kernel4x2(a + 4,         b + 4 * ld,     ld);
    kernel4x2(a + 4 * ld,     b + 4,         ld);
    kernel4x2(a + 4 * ld + 4, b + 4 * ld + 4, ld);
}

/* Этап 5: ядро 8x8 + блочный обход BS x BS */
void transpose_neon8_blocked(const double *restrict a, double *restrict b, size_t n,
size_t ld)
{
    size_t n8 = n / 8 * 8;
    for (size_t ii = 0; ii < n8; ii += BS)
        for (size_t jj = 0; jj < n8; jj += BS) {
            size_t ie = ii + BS < n8 ? ii + BS : n8;
            size_t je = jj + BS < n8 ? jj + BS : n8;
            for (size_t i = ii; i < ie; i += 8)
                for (size_t j = jj; j < je; j += 8)
                    kernel8(a + i * ld + j, b + j * ld + i, ld);
        }
    tail(a, b, n, ld, 8);
}

/* эталон: копирование того же объёма (практический предел памяти) */
void copy_ref(const double *restrict a, double *restrict b, size_t n, size_t ld)
{
    memcpy(b, a, n * ld * sizeof(double));
}

static size_t check_copy(const double *a, const double *b, size_t n, size_t ld)
{
    return memcmp(a, b, n * ld * sizeof(double)) != 0;
}

int main(int argc, char **argv)
{
    size_t n = arg_n(argc, argv, 8192);
    int reps = arg_reps(argc, argv, 5);
    size_t ld = arg_ld(argc, argv, n);
    double *a = alloc_matrix(n, ld), *b = alloc_matrix(n, ld);
    fill_matrix(a, n, ld);
    printf("N = %zu, BS = %d\n", n, BS);

    /* memcpy замеряем отдельно: bench() проверяет именно транспонирование */
    double tmin = 1e30;
    memset(b, 0, n * ld * sizeof(double));
    for (int r = 0; r < reps; r++) {
        double t0 = wtime();
        copy_ref(a, b, n, ld);
        double t = wtime() - t0;
        if (t < tmin) tmin = t;
    }
    printf("%-28s  min %8.4f s  %24.2f GB/s  %s\n", "memcpy (эталон)", tmin,
           2.0 * n * n * sizeof(double) / tmin / 1e9,
           check_copy(a, b, n, ld) ? "ОШИБКА" : "ok");

    bench("neon2 (2x2)", transpose_neon2, a, b, n, ld, reps);
    bench("neon2_blocked (2x2+BS)", transpose_neon2_blocked, a, b, n, ld, reps);
    bench("neon4_blocked (4x4+BS)", transpose_neon4_blocked, a, b, n, ld, reps);
    bench("neon4_pf (4x4 ldp/stp+pf)", transpose_neon4_pf, a, b, n, ld, reps);
    bench("neon8_blocked (8x8+BS)", transpose_neon8_blocked, a, b, n, ld, reps);
```

```
    free(a);
    free(b);
    return 0;
}
```

Листинг A.5 — transpose_sme.c. Векторизация на SME

```
/*
 * transpose_sme.c -- транспонирование на расширении SME (Scalable Matrix
 * Extension), самом новом векторном расширении, которое есть у Apple M4.
 *
 * В режиме streaming SVE у M4 длина вектора 512 бит (8 double), а
 * матричный регистр ZA делится на 8 тайлов ZA0.D..ZA7.D по 8x8 double.
 * Строку тайла можно загрузить "горизонтально" (svld1_hor_za64), а
 * сохранить "вертикально" (svst1_ver_za64) -- то есть столбец. Загрузив
 * строки A и выгрузив столбцы тайла, получаем транспонированный блок
 * без каких-либо перестановок в регистрах.
 *
 * Сборка: clang -O3 -mcpu=apple-m4 -DBS=256 transpose_sme.c
 * (-mcpu=native в Apple clang 17 SME не включает)
 * Запуск: ./transpose_sme [N] [повторы] [pad]
 */
#include "common.h"
#include <arm_sme.h>

#ifndef BS
#define BS 256              /* размер внешнего блока (тайла) */
#endif

static void tail(const double *a, double *b, size_t n, size_t ld, size_t k)
{
    size_t nk = n / k * k;
    for (size_t i = 0; i < n; i++)
        for (size_t j = (i < nk ? nk : 0); j < n; j++)
            b[j * ld + i] = a[i * ld + j];
}

/* длина вектора в режиме streaming SVE, в элементах double */
__arm_locally_streaming
static uint64_t sme_vl(void)
{
    return svcntd();
}

/*
 * Этап 1: блок 8x8 через один тайл ZA0: 8 горизонтальных загрузок
 * строк A, 8 вертикальных записей столбцов в строки B.
 */
__arm_locally_streaming __arm_new("za")
static void sme8_core(const double *a, double *b, size_t n8, size_t ld)
{
    svbool_t pg = svptrue_b64();
    for (size_t ii = 0; ii < n8; ii += BS)
        for (size_t jj = 0; jj < n8; jj += BS) {
            size_t ie = ii + BS < n8 ? ii + BS : n8;
            size_t je = jj + BS < n8 ? jj + BS : n8;
            for (size_t i = ii; i < ie; i += 8)
                for (size_t j = jj; j < je; j += 8) {
                    const double *pa = a + i * ld + j;
                    double *pb = b + j * ld + i;
                    for (uint32_t r = 0; r < 8; r++)
                        svld1_hor_za64(0, r, pg, pa + r * ld);
                    for (uint32_t c = 0; c < 8; c++)
                        svst1_ver_za64(0, c, pg, pb + c * ld);
                }
        }
}

void transpose_sme8(const double *restrict a, double *restrict b, size_t n, size_t ld)
{
```

```
    size_t n8 = n / 8 * 8;
    sme8_core(a, b, n8, ld);
    tail(a, b, n, ld, 8);
}

/*
 * Этап 2: блок 16x16 через четыре тайла ZA0..ZA3:
 *   ZA0 = A[0..7][0..7],  ZA1 = A[0..7][8..15],
 *   ZA2 = A[8..15][0..7], ZA3 = A[8..15][8..15].
 * Столбец с блока выгружается из ZA0/ZA2 (или ZA1/ZA3) двумя записями
 * подряд, то есть в B пишется сразу 16 double = 128 байт = целая строка
 * кэша M4, а из A читается тоже по 128 байт на строку.
 * Номер тайла в intrinsics должен быть константой, поэтому цикл по
 * тайлам расписан вручную.
 */
__arm_locally_streaming __arm_new("za")
static void sme16_core(const double *a, double *b, size_t n16, size_t ld)
{
    svbool_t pg = svptrue_b64();
    for (size_t ii = 0; ii < n16; ii += BS)
        for (size_t jj = 0; jj < n16; jj += BS) {
            size_t ie = ii + BS < n16 ? ii + BS : n16;
            size_t je = jj + BS < n16 ? jj + BS : n16;
            for (size_t i = ii; i < ie; i += 16)
                for (size_t j = jj; j < je; j += 16) {
                    const double *pa = a + i * ld + j;
                    double *pb = b + j * ld + i;
                    for (uint32_t r = 0; r < 8; r++) {
                        svld1_hor_za64(0, r, pg, pa + r * ld);
                        svld1_hor_za64(1, r, pg, pa + r * ld + 8);
                        svld1_hor_za64(2, r, pg, pa + (r + 8) * ld);
                        svld1_hor_za64(3, r, pg, pa + (r + 8) * ld + 8);
                    }
                    for (uint32_t c = 0; c < 8; c++) {
                        svst1_ver_za64(0, c, pg, pb + c * ld);
                        svst1_ver_za64(2, c, pg, pb + c * ld + 8);
                        svst1_ver_za64(1, c, pg, pb + (c + 8) * ld);
                        svst1_ver_za64(3, c, pg, pb + (c + 8) * ld + 8);
                    }
                }
        }
}

void transpose_sme16(const double *restrict a, double *restrict b, size_t n, size_t
ld)
{
    size_t n16 = n / 16 * 16;
    sme16_core(a, b, n16, ld);
    tail(a, b, n, ld, 16);
}

int main(int argc, char **argv)
{
    size_t n = arg_n(argc, argv, 8192);
    int reps = arg_reps(argc, argv, 5);
    size_t ld = arg_ld(argc, argv, n);
    if (sme_vl() != 8) {
        /* ядра написаны под тайл 8x8 double (SVL = 512 бит) */
        printf("длина вектора SME %llu double, ожидалось 8\n",
               (unsigned long long)sme_vl());
        return 1;
    }
    double *a = alloc_matrix(n, ld), *b = alloc_matrix(n, ld);
    fill_matrix(a, n, ld);
    printf("N = %zu, BS = %d, SVL = %llu бит\n", n, BS,
           (unsigned long long)sme_vl() * 64);
    bench("sme8 (ZA 8x8+BS)", transpose_sme8, a, b, n, ld, reps);
    bench("sme16 (4 tiles ZA 16x16+BS)", transpose_sme16, a, b, n, ld, reps);
    free(a);
    free(b);
    return 0;
}
```